3️⃣

Phase 3 - Tokyo Completion Code (Sanitized)

This phase completes Tokyo by adding TGW peering to São Paulo, return routes, and the RDS security group update.

⚠️ **DEPENDENCY:** Phase 3 can only run AFTER Phase 2 completes. You need the São Paulo TGW ID.

CIDR Reference

Region	Resource	CIDR
Tokyo	VPC	`10.0.0.0/16`
Tokyo	Private Subnets	`10.0.11.0/24`, `10.0.12.0/24`
São Paulo	VPC	`10.1.0.0/16`
São Paulo	Private Subnets	`10.1.11.0/24`, `10.1.12.0/24`

Step 1: Update `terraform.tfvars` with São Paulo TGW ID

File: `lab3/tokyo/terraform.tfvars`

ADD:

```
# From Phase 2 output: terraform output -raw saopaulo_tgw_id
saopaulo_tgw_id   = "tgw-XXXXXXXXXXXXXXXXX"  # REPLACE with a
```

```
ctual
saopaulo_vpc_cidr = "10.1.0.0/16"
```

Step 2: Add TGW Peering to `04-3a-tokyo-tgw.tf`

ADD to existing file:

```
##############################################
# TGW PEERING ATTACHMENT (Tokyo → São Paulo)
# Shinjuku opens corridor to Liberdade
# Tokyo (10.0.0.0/16) ↔ São Paulo (10.1.0.0/16)
##############################################

resource "aws_ec2_transit_gateway_peering_attachment" "shinju
ku_to_liberdade_peer01" {
  count                   = var.enable_tgw && var.saopaulo_tg
w_id != "" ? 1 : 0
  transit_gateway_id      = aws_ec2_transit_gateway.shinjuku_
tgwlab3[0].id
  peer_region             = "sa-east-1"
  peer_transit_gateway_id = var.saopaulo_tgw_id

  tags = {
    Name = "shinjuku-to-liberdade-peer01"
    Type = "Cross-Region-Peering"
  }
}

##############################################
# TGW ROUTE TABLE ENTRY FOR PEERING
# Route to São Paulo CIDR (10.1.0.0/16)
##############################################

resource "aws_ec2_transit_gateway_route" "shinjuku_route_to_l
iberdade" {
```

```
  count                           = var.enable_tgw && var.saop
aulo_tgw_id != "" ? 1 : 0
  destination_cidr_block          = var.saopaulo_vpc_cidr  # 1
0.1.0.0/16
  transit_gateway_attachment_id  = aws_ec2_transit_gateway_pe
ering_attachment.shinjuku_to_liberdade_peer01[0].id
  transit_gateway_route_table_id = aws_ec2_transit_gateway.sh
injuku_tgwlab3[0].association_default_route_table_id
}
```

Step 3: Create `04-3b-tokyo-routes.tf` (NEW FILE)

What it does: Adds return route so Tokyo VPC can send traffic back to São Paulo.

```
##############################################
# TOKYO RETURN ROUTES TO SÃO PAULO
# Adds route in helga_private_rtlab3
# Destination: 10.1.0.0/16 → TGW
##############################################

resource "aws_route" "shinjuku_to_sp_route01" {
  count = var.enable_tgw ? 1 : 0

  # References YOUR Lab 1C route table
  route_table_id         = aws_route_table.helga_private_rtla
b3.id
  destination_cidr_block = var.saopaulo_vpc_cidr  # 10.1.0.0/
16
  transit_gateway_id     = aws_ec2_transit_gateway.shinjuku_t
gwlab3[0].id
}
```

Step 4: Create `04-3c-rds-sg-update.tf` (NEW FILE)

What it does: Allows São Paulo VPC CIDR to access your `helga_rds01` on port 3306.

```
##############################################
# RDS SECURITY GROUP RULE FOR SÃO PAULO
# Allows São Paulo compute (10.1.0.0/16) to access helga_rdslab3
##############################################

resource "aws_security_group_rule" "shinjuku_rds_ingress_from_liberdade01" {
  count = var.enable_tgw ? 1 : 0

  type              = "ingress"
  from_port         = 3306
  to_port           = 3306
  protocol          = "tcp"
  cidr_blocks       = [var.saopaulo_vpc_cidr]  # 10.1.0.0/16
  description       = "MySQL from Sao Paulo VPC via TGW"

  # References YOUR Lab 1C RDS security group
  security_group_id = aws_security_group.helga_rds_sglab3.id
}
```

🔐 **SECURITY NOTE:** This rule enables cross-region DB access while maintaining APPI compliance (data stays in Tokyo). Only the São Paulo CIDR `10.1.0.0/16` can access RDS.

Deployment Commands (Phase 3)

```
cd lab3/tokyo

# Plan (should show: peering attachment, TGW route, VPC rout
```

```
e, RDS SG rule)
terraform plan -out=phase3.tfplan

# Review the plan carefully - confirm:
# - 1 TGW peering attachment
# - 1 TGW route (destination: 10.1.0.0/16)
# - 1 VPC route in helga_private_rtlab3 (destination: 10.1.0.
0/16)
# - 1 SG rule in helga_rds_sglab3 (source: 10.1.0.0/16, port
3306)

# Apply
terraform apply phase3.tfplan
```

Verification Checklist (Phase 3)

- ☐ TGW peering attachment created (state: `pendingAcceptance`)
- ☐ TGW route table has entry for São Paulo CIDR (`10.1.0.0/16`)
- ☐ `helga_private_rtlab3` has route to `10.1.0.0/16` via TGW
- ☐ `helga_rds_sglab3` has inbound rule for `10.1.0.0/16` on port 3306

```
# Check peering state
aws ec2 describe-transit-gateway-peering-attachments --region
ap-northeast-1 \
  --query "TransitGatewayPeeringAttachments[*].{ID:TransitGat
ewayAttachmentId,State:State}"

# Expected: State = "pendingAcceptance"

# Check RDS SG rules (should see 10.1.0.0/16 on port 3306)
aws ec2 describe-security-groups --region ap-northeast-1 \
  --filters "Name=group-name,Values=*rds*" \
  --query "SecurityGroups[*].IpPermissions[?FromPort==\`3306
\`]"
```

```
# Check VPC route table for São Paulo route
aws ec2 describe-route-tables --region ap-northeast-1 \
  --filters "Name=tag:Name,Values=*helga-private*" \
  --query "RouteTables[*].Routes[?DestinationCidrBlock=='10.
1.0.0/16']"
```

🔗 **Next Step:** Peering is in `pendingAcceptance`. Proceed to Phase 4 to accept from São Paulo.